Lab 7 – Fuzz Testing for Input Validation Bugs

Name: Diya B R
SRN: PES1UG24AM097
Section: B
Semester: 5

Subject: Software Engineering Lab
Experiment: Fuzz Testing Using Hypothesis
Programming Language: Python
Testing Frameworks: Hypothesis and pytest

---

1. Objective

To explore fuzz testing as a software testing technique by identifying and fixing bugs in a Python application using the Hypothesis property-based fuzzing library.

The experiment focuses on testing functions with unexpected and invalid inputs to improve application robustness.

2. Software Requirements

- Python 3.8 or above
- Hypothesis
- pytest

3. Introduction

Fuzz testing is a dynamic testing technique in which random, unexpected, or malformed inputs are supplied to a program to identify bugs, crashes, and unexpected behaviour.

Hypothesis is a Python library used for property-based testing. It automatically generates different inputs and helps identify edge cases that may not be covered by manually written test cases.

4. Application Description

The assigned application is a Weather Alert Classifier containing three functions:

1. **parse_reading(text):** Parses temperature and humidity values from a string separated by `/`.
2. **wind_category(speed_text):** Classifies wind speed into warning, advisory, or none.
3. **rainfall_spread(readings_csv):** Calculates the difference between the maximum and minimum rainfall readings.

5. Execution of Original Application

The original application was executed using sample inputs:

- Temperature/Humidity: `35.5/80`
- Wind Speed: `45`
- Rainfall Readings: `2,5.5,10`

The application successfully processed the sample inputs and displayed the corresponding results.

```
(venv) root@Disha:/mnt/c/Users/50207/OneDrive/Desktop/Diya B R/Sem 5/SE/lab-6/L6_L7_32_weather_alert/lab7# pyth
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): 35.5/80
Temperature and Humidity: (35.5, 80)

==== Running wind_category ====
Enter a wind speed in km/h (e.g. 45): 45
Wind Category: advisory

==== Running rainfall_spread ====
Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): 2,5.5,10
(venv) root@Disha:/mnt/c/Users/50207/OneDrive/Desktop/Diya B R/Sem 5/SE/lab-6/L6_L7_32_weather_alert/lab7#
```

6. Implementation of Fuzz Testing

Hypothesis was used to generate random strings and `None` values for all three functions.

The following test functions were created:

- `test_parse_reading_no_crash()`
- `test_wind_category_no_crash()`
- `test_rainfall_spread_no_crash()`

The tests were executed using:

```
python -m pytest test_weather_alert.py -v
```

Initial Test Results

Hypothesis identified failures in two functions.

```
rootdir: /mnt/c/Users/50207/OneDrive/Desktop/Diya B R/Sem 5/SE/lab-6/L6_L7_32_weather_alert/lab7
plugins: hypothesis-6.170.15
collected 3 items

test_weather_alert.py::test_parse_reading_no_crash FAILED            [ 33%]
test_weather_alert.py::test_wind_category_no_crash PASSED            [ 66%]
test_weather_alert.py::test_rainfall_spread_no_crash FAILED          [100%]

================================== FAILURES ===================================
_____________________ test_parse_reading_no_crash _____________________

    @given(st.text() | st.none())
>   def test_parse_reading_no_crash(s):
                ^^^

test_weather_alert.py:7:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
test_weather_alert.py:9: in test_parse_reading_no_crash
    parse_reading(s)
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

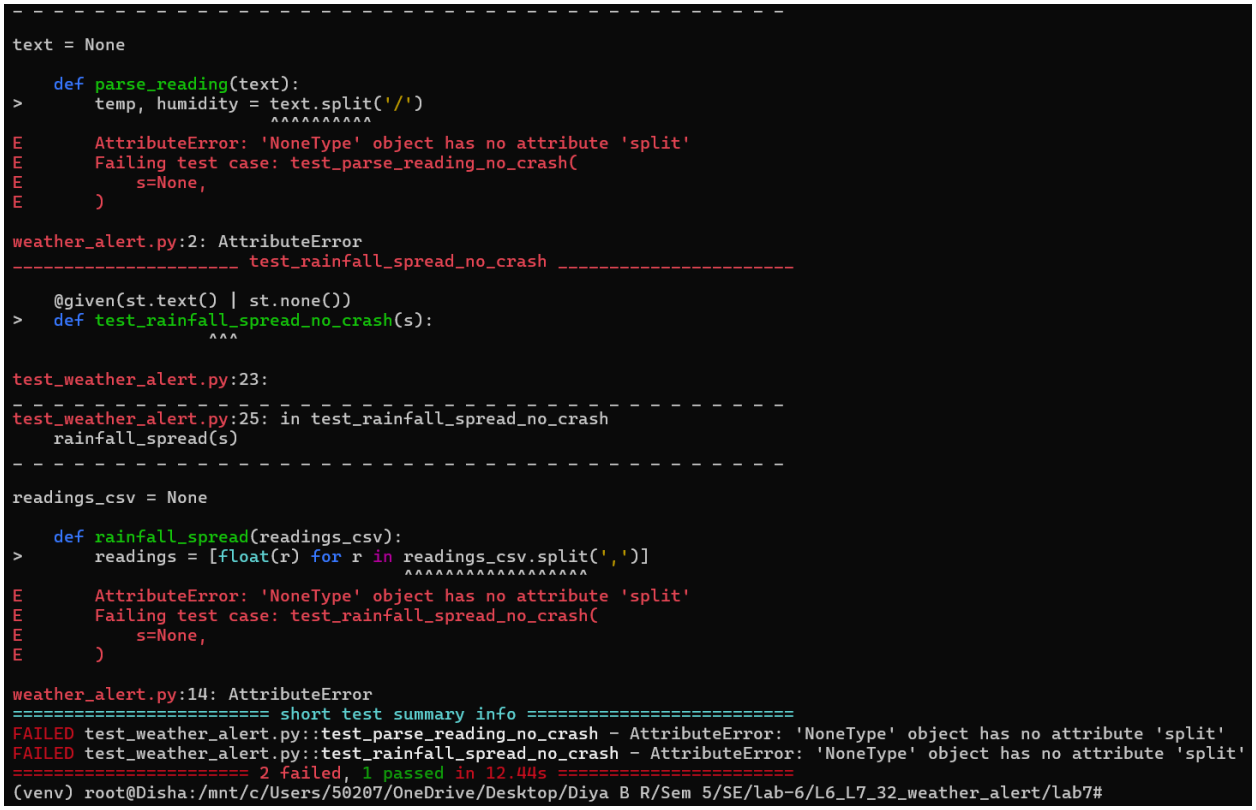
text = None

    def parse_reading(text):
>       temp, humidity = text.split('/')
                         ^^^^^^^^^^
E       AttributeError: 'NoneType' object has no attribute 'split'
E       Failing test case: test_parse_reading_no_crash(
E           s=None,
E       )

weather_alert.py:2: AttributeError
_____________________ test_rainfall_spread_no_crash _____________________

    @given(st.text() | st.none())
>   def test_rainfall_spread_no_crash(s):
                ^^^

test_weather_alert.py:23:
```



Analysis of Identified Bugs

1. parse_reading():

- Failing input: `None`
- The function attempted to call `.split('/')` on a None value.
- This resulted in an `AttributeError`.

2. rainfall_spread():

- Failing input: `None`
- The function attempted to call `.split(',')` on a None value.
- This also resulted in an `AttributeError`.

3. wind_category():

- The function passed the generated fuzz test cases without an unexpected exception.

Initial test summary: **2 failed, 1 passed.**

7. Bug Fixes and Code Improvements

The application was modified to handle invalid inputs more safely.

The following improvements were implemented:

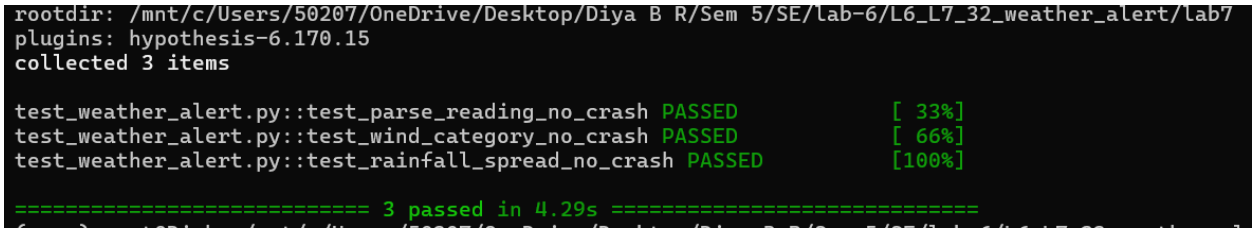
- Added type validation to ensure that inputs are strings.
- Added validation for the temperature and humidity input format.
- Added exception handling for invalid numeric values.
- Added validation for empty rainfall input.
- Added handling for invalid rainfall values.
- Invalid inputs now produce meaningful `ValueError` exceptions instead of unexpected attribute errors.

8. Execution of Fixed Application

After modifying the application, it was executed again using the same sample inputs.

The application produced the following results:

- Temperature and Humidity: `(35.5, 80)`
- Wind Category: `advisory`
- Rainfall Spread: `8.0`



9. Final Fuzz Testing Results

The Hypothesis tests were executed again after fixing the application.

Command:

```
python -m pytest test_weather_alert.py -v
```

All three test functions passed successfully.

```
(venv) root@Disha:/mnt/c/Users/50207/OneDrive/Desktop/Diya B R/Sem 5/SE/lab-6/L6_L7_32_weather_alert/lab7# python weather_alert.py
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): 35.5/80
Temperature and Humidity: (35.5, 80)

==== Running wind_category ====
Enter a wind speed in km/h (e.g. 45): 45
Wind Category: advisory

==== Running rainfall_spread ====
Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): 2,5.5,10
(venv) root@Disha:/mnt/c/Users/50207/OneDrive/Desktop/Diya B R/Sem 5/SE/lab-6/L6_L7_32_weather_alert/lab7#
```

Final test summary: **3 passed.**

10. Conclusion

This experiment demonstrated how fuzz testing can identify unexpected input-handling bugs that may not be discovered through normal manual testing.

Using Hypothesis, failures were identified in the `parse_reading()` and `rainfall_spread()` functions when `None` was supplied as input.

The application was improved through input validation and exception handling. After the modifications, all three fuzz tests passed successfully.

The experiment helped demonstrate the importance of property-based testing and defensive programming in developing robust software applications.

Deliverables:

1. PDF report containing execution and testing screenshots.
2. Fixed `weather_alert.py` application.
3. Updated `test_weather_alert.py` test file.